The Midnight Queue: Ranking Suspicious Transactions at Scale

Assignment 2, Design & Analysis of Algorithms | **K = 100** (shift capacity) | all randomness from one fixed seed

Setup. Timings in this report were measured on a cloud VM: Intel Xeon 2.8 GHz (2 vCPU), 7 GB RAM, Linux, Python 3.13 (NumPy/pandas only for loading and the risk score). I also ran the code on my laptop (AMD Ryzen 5 7535HS, 8 GB RAM, Windows 64-bit) to double-check the results. Risk $= \sum z_i^2$ with z_i standardised over the full file; band $= \min(9, \max(0, \lfloor \log_2 \text{risk} \rfloor))$. **Sort key:** the triage order (band↓, Amount↓, Time↑, txn_id↑) is packed into one exact integer, so every comparison is a single < between two ints. **Counting:** a *comparison* is one key-vs-key test; a *move* is one write of a key into a list slot (a swap = 2 moves); *extra* is the peak number of extra slots used. **Timing:** `perf_counter`, a fresh copy of the input for every run, median of 5 runs. A run over 5 minutes is recorded as DNF, and larger n for that contender are then not run.

A. Know your data

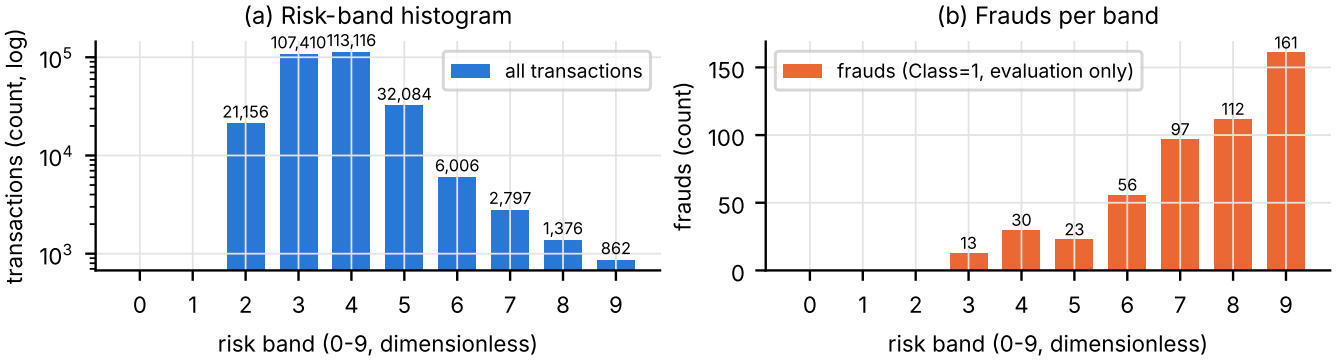


Figure 1: Risk bands of all 284,807 transactions. (a) Count per band (log scale); bands 0–1 are empty because the minimum risk is 4.21. (b) Frauds per band (Class is used for evaluation only).

- **Bands:** 2: 21,156 | 3: 107,410 | 4: 113,116 | 5: 32,084 | 6: 6,006 | 7: 2,797 | 8: 1,376 | 9: 862. Bands 0–1 are empty, and bands 3 and 4 hold 77% of all records.
- **Amount:** 32,767 distinct values; the 10 most frequent (1.00, 1.98, 0.89, 9.99, 15.00, ...) cover **16.3%** of rows. 88% of records share their (band, Amount) pair with another record, so ties are very common.
- **Frauds:** 492, by band 9: 161, 8: 112, 7: 97, 6: 56, 5: 23, 4: 30, 3: 13. Bands 6–9 are only 3.9% of rows but hold 87% of the frauds.
- **Properties of the raw file:** it is in Time order (C1). For the triage key it has 50.06% of the maximum possible inversions, so it behaves like a **random shuffle (C2)**, not like C3, C4 or C5. With the band-only key (C6) it has just 8 distinct values, so it is heavily tied.

B. The contenders and their correctness

All five sorts and both top-K methods are written by hand in `algos.py`, with no built-in sort inside them. Naive Quick Sort uses the last element as pivot with Lomuto partition. My **Hardened Quick Sort uses a random pivot plus three-way partition**: the random pivot fixes sorted and reverse input, and three-way partition fixes the many ties (only 8 band values). **Tests** (`tests.py`, all pass) compare every contender with Python's `sorted()` on empty input, a single record, all-equal keys, duplicate-heavy keys, sorted, reversed and random input, and all six conditions. In addition, every one of the 108 completed timed runs gave output identical to the reference. Insertion and Merge Sort are stable; the Quick Sorts and Heap Sort are not. **Failures:** a recursive Naive Quick Sort would crash Python's recursion limit on sorted input, so mine uses an explicit stack (smaller side first). Its real failure is time: it DNFs on C3, C4 and C6, and Insertion Sort DNFs on C1, C2, C4 and C6.

C. The full ranking tournament

Table 1: Summary for every contender and condition: median time (s) at $n = 10^4$, 10^5 and 284,807; comparisons and moves at $n = 10^4$; peak extra slots at the largest completed n; stability (st.). DNF = a single run exceeded 300 s.

	Contender	t_{10k}	t_{100k}	t_{285k}	comp_{10k}	mov_{10k}	extra	st.
C1	Insertion	4.160	**DNF**	**DNF**	24.61M	24.61M	0	yes
	Merge	0.024	0.323	1.106	120.2k	190.3k	142.4k	yes
	Naive Quick	0.028	0.441	1.362	162.0k	191.7k	13	no
	Hardened Quick	0.041	0.559	2.035	248.9k	327.6k	12	no
	Heap	0.030	0.541	1.706	235.4k	149.1k	0	no
C2	Insertion	4.941	**DNF**	**DNF**	25.16M	25.16M	0	yes
	Merge	0.025	0.390	1.305	120.5k	190.5k	142.4k	yes
	Naive Quick	0.024	0.411	1.311	150.0k	169.1k	13	no
	Hardened Quick	0.042	0.660	2.108	235.8k	313.5k	13	no
	Heap	0.029	0.537	2.171	235.4k	149.2k	0	no
C3	Insertion	1.97ms	0.034	0.124	10.0k	10.0k	0	yes
	Merge	0.014	0.190	0.624	64.6k	129.2k	142.4k	yes
	Naive Quick	9.200	**DNF**	**DNF**	49.99M	100.01M	2	no
	Hardened Quick	0.041	0.687	2.236	227.6k	303.5k	12	no
	Heap	0.031	0.459	1.435	244.5k	157.0k	0	no

	Contender	t_{10k}	t_{100k}	t_{285k}	comp_{10k}	mov_{10k}	extra	st.
C4	Insertion	8.618	**DNF**	**DNF**	49.99M	50.00M	0	yes
	Merge	0.022	0.343	1.129	69.0k	198.2k	142.4k	yes
	Naive Quick	6.504	**DNF**	**DNF**	49.99M	50.01M	2	no
	Hardened Quick	0.040	0.631	2.026	237.0k	305.5k	12	no
	Heap	0.028	0.487	1.353	226.7k	141.7k	0	no
C5	Insertion	0.055	8.146	68.0	355.3k	355.3k	0	yes
	Merge	0.019	0.279	1.004	91.4k	156.2k	142.4k	yes
	Naive Quick	0.323	7.345	25.8	2.53M	799.8k	8	no
	Hardened Quick	0.042	0.694	2.238	237.2k	325.4k	13	no
	Heap	0.031	0.450	1.417	244.9k	156.7k	0	no
C6	Insertion	2.310	242.0	**DNF**	17.06M	17.06M	0	yes
	Merge	0.019	0.232	0.735	114.5k	181.9k	142.4k	yes
	Naive Quick	2.354	251.1	**DNF**	16.00M	32.00M	7	no
	Hardened Quick	4.06ms	0.036	0.130	35.7k	21.7k	4	no
	Heap	0.021	0.269	0.867	188.3k	120.4k	0	no

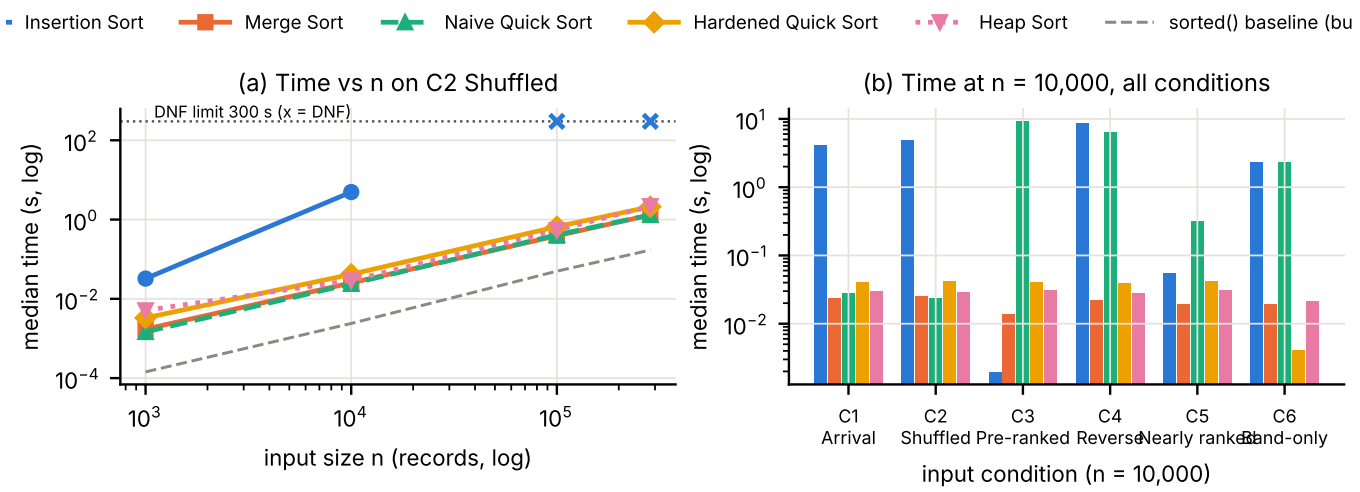


Figure 2: (a) Median execution time vs n on C2 Shuffled (log–log); $\times$ marks a DNF (single run > 300 s); the dashed grey line is the built-in `sorted()` baseline (C Timsort, not a contender). (b) Median time at $n = 10{,}000$ for all six input conditions (log scale).

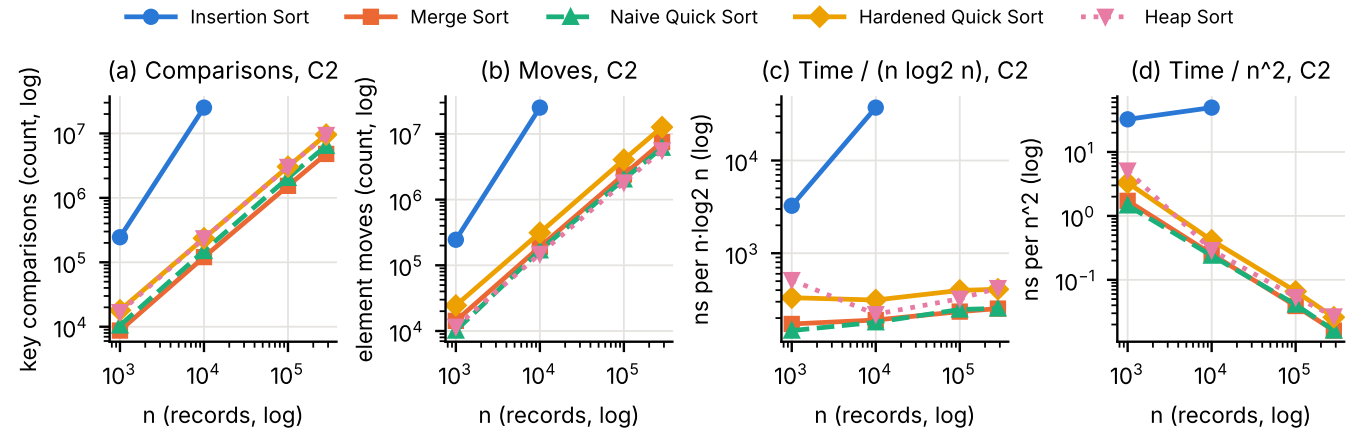


Figure 3: C2 Shuffled. (a) Key comparisons and (b) element moves vs n (log–log). (c) Time$/(n \log_2 n)$ and (d) Time$/n^2$, in nanoseconds, vs n: a flat line means that growth model fits. Insertion Sort is flat only in (d); the $n \log n$ sorts are near-flat in (c) and fall in (d).

Results by condition (full n unless stated). C1/C2 random: Merge Sort (1.11 / 1.31 s) and Naive Quick (1.36 / 1.31 s) are fastest; Heap and Hardened Quick take about 2 s; Insertion DNFs from $n = 10^5$. **C3 pre-ranked:** Insertion wins (0.12 s, only $n - 1$ comparisons); Naive Quick DNFs from 10^5. **C4 reverse:** Merge wins (1.13 s); Insertion and Naive Quick DNF from 10^5. **C5 nearly ranked:** Insertion wins only at $n = 1000$; at full n it takes 68 s against Merge's 1.0 s. **C6 band-only:** Hardened Quick wins easily (0.13 s); Naive Quick and Insertion become quadratic (about 250 s at 10^5, DNF at full n).

Predicted DNF points ($t \propto n^2$ from the largest finished n): Insertion on C2 ≈ 494 s at 10^5 and 4,008 s at full n; Naive Quick on C3 ≈ 920 s and 7,463 s. All are well over 300 s. Checked one scale lower, the same model predicted the 10^4 times within 5–34% (always an underestimate).

Clash: Hardened Quick Sort has the same $O(n \log n)$ as Naive Quick but is 1.6$\times$ slower on random data (2.11 vs 1.31 s), because three-way partition does extra comparisons. I still choose it, because Naive Quick fails on 3 of the 6 conditions. **Clash:** In theory time$/(n \log n)$ should stay flat, but it rises about 1.5$\times$ as n grows (Fig. 3c), due to memory and cache effects in Python. This is small compared with Insertion Sort's n^2 growth, so I still treat these sorts as $n \log n$. Python's built-in `sorted()`, shown only as a baseline, is about 8$\times$ faster than all contenders because it runs in C.

D. The shift queue

Where each method stops being best: the **size-K heap was fastest at every K tested**, on C1 and C2 at both sizes (e.g. 0.03 s vs 1.05 s for a full sort at $K = 100$, full n). The **buffer** is close to the heap for small K but grows like K^2: it is slower than a full sort somewhere between $K = 500$ and 5,000, and DNFs at $K = n/10$. A **full sort** only beats the heap when K is very large; an extra test at $n = 10^5$ put the crossover between $K = 20{,}000$ and 50,000. **Fraud capture:** the smallest K that captures half of the frauds is $K = 2{,}068$. Building that queue on the full file takes 0.06–0.11 s with the heap, 1.3–1.4 s with a full sort and 1.9 s with the buffer. **Clash:** my $K = 100$ queue contains **0 frauds**: the top 100 are band-9 transactions with very large Amounts, and every band-9 fraud is smaller (at most 1,218.89). The fastest algorithm does not fix a key that ranks the wrong records first. I keep the required triage order and point this out in H.

E. The live feed

Totals (whole 48 h): with 1-hour windows, re-rank takes 29.2 s, the size-K heap 0.03 s and sort-window-and-merge 1.09 s. With 10-minute windows they take 135.6 s, 0.06 s and 0.56 s. **Peak extra memory (slots):** re-rank 712,018, because it keeps every record; heap 200; merge 15,043 for 1-hour and 3,529 for 10-minute windows. Smaller windows make re-ranking much worse

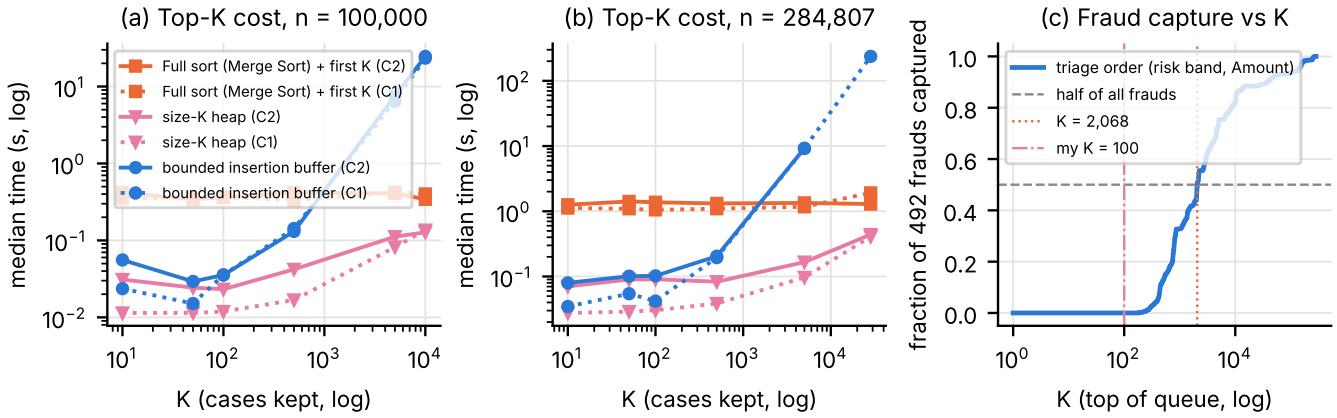


Figure 4: (a, b) Median time of the three top-K methods vs K at $n = 100{,}000$ and 284,807 (solid C2 Shuffled, dotted C1 Arrival; log–log). The buffer's missing point at $K = 28{,}480$ on C2 is a DNF. (c) Fraction of the 492 frauds inside the top K (Class used for evaluation only).

Table 2: Top-K on C2: median time (fastest in bold) and comparisons. Full sort = Merge Sort, then take the first K.

		median time (s), C2 Shuffled			comparisons, C2		
n	K	full sort	size-K heap	buffer	full sort	size-K heap	buffer
100,000	10	0.418	**0.031**	0.056	1.54M	100.5k	100.5k
100,000	50	0.341	**0.024**	0.029	1.54M	103.4k	109.8k
100,000	100	0.360	**0.023**	0.035	1.54M	107.9k	135.2k
100,000	500	0.421	**0.042**	0.131	1.54M	146.6k	813.8k
100,000	5,000	0.408	**0.112**	7.567	1.54M	534.4k	43.78M
100,000	10,000	0.342	**0.128**	24.6	1.54M	884.5k	140.69M
284,807	10	1.260	**0.071**	0.080	4.80M	285.4k	285.5k
284,807	50	1.415	**0.090**	0.101	4.80M	289.0k	296.8k
284,807	100	1.374	**0.091**	0.102	4.80M	294.2k	327.6k
284,807	500	1.319	**0.083**	0.204	4.80M	339.3k	1.13M
284,807	2,068	1.424	**0.110**	1.919	4.80M	519.0k	11.95M
284,807	5,000	1.343	**0.166**	9.253	4.80M	828.5k	56.84M
284,807	28,480	1.291	**0.439**	DNF	4.80M	2.79M	–

(4.6×), make (c) slightly cheaper and barely affect the heap, so the heap is best at both window sizes. **Clash:** the brief asks for the median of 5 runs, but because re-ranking at 10-minute windows takes minutes per run, Problem E used the median of 3. The gap between strategies is so large (over 900×) that this cannot change the result.

F. The mathematical case

M1 Insertion Sort. Each element moves left past every larger element before it, so the work is the number of inversions I: $T(n) = \Theta(n+I)$, with comparisons $\approx I + (n-1)$. **Best** (sorted, $I = 0$): $n-1$ comparisons, $\Theta(n)$. **Worst** (reversed, $I = n(n-1)/2$): $\Theta(n^2)$. **Average** (random, $I \approx n^2/4$): $\Theta(n^2)$. On my C5 sample $I =$ **276,109,519** at full n (33,261,300 at 10^5), and the measured comparisons equal $I + (n-1)$ exactly. **Clash:** "nearly ranked" sounds easy for Insertion Sort, but moving 1% of records to random places still creates about $0.0033n^2$ inversions, so it stays quadratic. This is why Insertion loses C5 at large n.

M2 Merge Sort. Iteration: $T(n) = 2T(n/2) + n = 4T(n/4) + 2n = 8T(n/8) + 3n = \cdots = 2^k T(n/2^k) + kn$. Stopping at $n/2^k = 1$, i.e. $k = \log_2 n$, gives $T(n) = n + n\log_2 n = \Theta(n \log n)$. Master theorem: $a = 2$, $b = 2$, $n^{\log_2 2} = n$ and $f(n) = \Theta(n)$, so case 2 gives $\Theta(n \log n)$. Measured comparisons at full n: 4,804,255, close to $n \log_2 n$.

M3 Quick Sort. (a) Worst case by iteration: $T(n) = T(n-1) + n = T(n-2) + (n-1) + n = \cdots = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} = \Theta(n^2)$. On C3, Naive Quick made exactly $n(n-1)/2$ comparisons (49,995,000 at $n = 10^4$). (b) Best case by recursion tree: every level has total cost n and there are $\log_2 n$ levels, so $T(n) = \Theta(n \log n)$. (c) 1:9 split by recursion tree: every level still costs at most n, and the deepest path has $\log_{10/9} n$ levels (still a logarithm), so $T(n) = \Theta(n \log n)$.

M4 Heap Sort. Building the heap is $O(n)$, because most nodes are near the bottom and sift down only a few levels. Then n times we remove the maximum and sift down at most $\log_2 n$ levels, which costs $O(n \log n)$. Total: $\Theta(n \log n)$ for any input order.

M5 Top-K. (a) Full sort, then take K: $O(n \log n)$, whatever K is. (b) Size-K heap: each record is compared once with the root, and only the few that get in cost $\log K$, so $O(n \log K)$ in the worst case and close to $O(n)$ for small K. (c) Buffer: each record that gets in shifts up to K elements, so $O(nK)$ in the worst case. **Prediction** for $n = 10^5$: a heap insert costs about $2\log_2 K$ against about $K/2$ for the buffer, so the buffer should win only for K below about 16. **Measured:** at $K = 10$ the comparison counts are almost equal, but the heap was faster in time at every K. **Clash:** the comparison count says the buffer should win at $K = 10$, but the timing says the heap wins, because the buffer's loop has extra Python overhead per record. I go with the measurement.

M6 Naive Quick Sort on C6. With Lomuto (<=) and the last pivot, a block of m equal keys always splits into $m-1$ and 0, costing $(m-1) + (m-2) + \cdots + 1 = \frac{m(m-1)}{2}$. So comparisons $\approx \sum_i \frac{m_i(m_i-1)}{2}$ over the band groups. My groups at $n = 10^5$ (bands 2–9): 7,351 / 37,685 / 39,733 / 11,316 / 2,110 / 985 / 492 / 328. Predicted **1,593,315,772**, measured **1,593,506,374**, within 0.012%.

M7 Live feed (w windows). (a) Re-ranking everything after each window costs about $w \cdot n \log n$. (b) The size-K heap costs about $n \log K$ in total. (c) Sorting each window and merging costs about $n \log(n/w) + wK$. (b) is the cheapest unless the windows

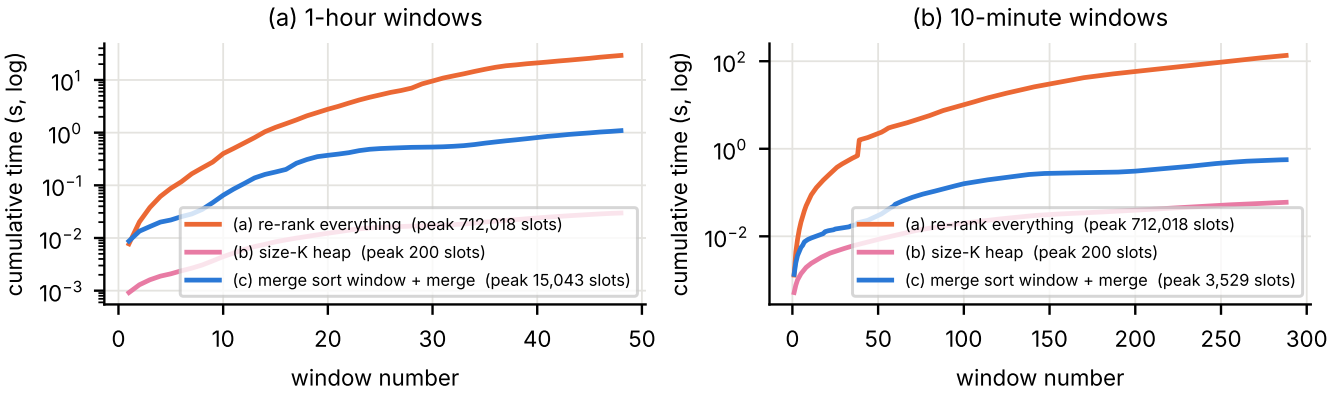


Figure 5: Cumulative time (s, log scale) against window number while publishing the top $K = 100$ after every window, for (a) 48 one-hour windows and (b) 288 ten-minute windows. Median of 3 runs per window index. The legend gives each strategy's peak extra memory in key slots. All $48 + 288$ published lists were checked against the reference.

become tiny (about 1-minute windows for $K = 100$); then (c) takes over. (a) is never best.

M8 Space. Insertion $O(1)$; Merge $O(n)$ (a buffer of $n/2$); Naive and Hardened Quick $O(\log n)$ stack; Heap Sort $O(1)$; size-K heap and buffer $O(K)$; live feed (a) $O(n)$, (b) $O(K)$, (c) $O(n/w + K)$.

H. Recommendation (to the platform lead)

1. Overnight batch ⇒ Merge Sort. *Evidence:* fastest or tied fastest on C1, C2, C4 and C5 at full n, and never slower than 1.31 s on any condition, while Naive Quick and Insertion DNF on several. It is guaranteed $O(n \log n)$ (M2) and stable. *Assumption:* we can spare $n/2$ extra memory and we sort on the full triage key. *Wrong if:* we sort on band only (C6), where Hardened Quick is 5.6× faster, or memory is very tight (use Heap Sort).

2. Shift queue of K cases ⇒ size-K heap. *Evidence:* fastest at every K tested: 0.03 s vs 1.05 s for a full sort at $K = 100$ (35×), with cost close to $O(n)$ (M5). *Assumption:* K is much smaller than n. *Wrong if:* K gets close to n (beyond about $n/3$ a full sort is faster). *Warning:* with $K = 100$ the queue holds 0 of the 492 frauds; half of them need $K = 2{,}068$, which the heap still builds in about 0.1 s.

3. Live refresh ⇒ persistent size-K heap. *Evidence:* 0.03 s / 0.06 s for the whole 48 h (1-hour / 10-minute windows), against 29 s / 136 s for re-ranking, using only 200 slots of memory (M7). *Assumption:* a transaction's score never changes after it arrives. *Wrong if:* scores are updated or records deleted, since the heap has already thrown away everything below the top K (then re-ranking is needed), or windows become tiny (about 1 minute), where (c) wins.

Bonus: The Analyst Console

`console.html` is a single self-contained web page; it opens in any browser with no server. It contains my contenders ported line-for-line from `algos.py`, with the same comparison and move counting, and all 284,807 transactions keyed by their triage rank. The analyst picks the algorithm that builds the queue (size-K heap, insertion buffer, or any of the five full sorts followed by the first K), sets K (default 100), the condition (C1–C5) and n. The page shows the top K with txn_id, risk band, Amount and Time, plus the running time, comparisons, moves, peak extra memory and a check against the reference. A history table and a "Run all algorithms" button put every method side by side, for example Naive Quick collapsing on C3, or the heap needing about 15k comparisons against 25M for Insertion. Runs over 20 s show as DNF. (A Python version that calls `algos.py` directly is in `console.py`.)

AI-use disclosure

I used Claude (Anthropic) to build the measurement harness, data loading, tests, plotting and table scripts, the notebook and the bonus console, and for help drafting and typesetting this report, including the derivation write-ups and the wording of the prediction log. The derivations follow my algorithms course notes. The contenders in `algos.py` are my own code. Every number in this report comes from `results.csv`, produced by running this code, and I checked the derivations against the measured counts. I also ran the code on my own machine to double-check the results.

Appendix: Prediction log

Round	Prediction (written before the run)	What happened / what I got wrong
C1 Arrival	Naive Quick wins (arrival is random for my key, which suits a last pivot). Insertion loses: DNF at 10^5 ($\approx$450 s by my estimate). Heap $\approx$ Merge (both $n \log n$).	Merge narrowly beat Naive Quick at full n (1.11 vs 1.36 s); Insertion DNF ✓ (416 s predicted). **Wrong:** Heap was 1.5× slower than Merge; its sift-down makes $\approx 2n \lg n$ comparisons vs Merge's $n \lg n$.
C2 Shuffled	Naive Quick wins; Hardened Quick a close 2nd ("one random number per partition"); Merge makes the fewest comparisons.	Naive Quick = Merge (1.31 s) ✓; Merge fewest comparisons ✓. **Wrong:** Hardened Quick was 1.6× slower. The cost is not the random number but the three-way partition's 2nd comparison (236k vs 150k).
C3 Pre-ranked	Insertion wins ($n-1$ comparisons); Naive Quick DNF at 10^5; Heap same as C2; Merge a bit faster.	All ✓; Merge did half the comparisons (64.6k vs 120k). **Wrong:** Heap was 34% *faster* than on C2 (1.44 vs 2.17 s). Order does not change its Θ, but it changes how far keys sift and how cache-friendly the access is.
C4 Reverse	Insertion and Naive Quick both DNF; Merge wins.	All ✓ (Merge 1.13 s, Heap 2nd). Both losers did exactly $\binom{n}{2}$ comparisons at 10^4, matching M1/M3.
C5 Nearly ranked	Insertion wins at *all* sizes (99% in place); Naive Quick behaves like $n \log n$.	**Wrong twice.** Insertion wins only at $n = 1000$; at full n it takes 68 s vs Merge's 1.0 s, because $I \approx 0.0033n^2$ (M1). Naive Quick took 25.8 s (20× its C2 time): 1% movers do not rescue a last pivot on long sorted runs.
C6 Band-only	Everyone faster (8 keys); Hardened Quick wins; Naive Quick fine (C1 order is random); Insertion DNF at 10^5.	Hardened Quick wins ✓ (0.13 s, 5.6× ahead). **Wrong:** Naive Quick became quadratic (251 s at 10^5, DNF at full n). Ties, not order, defeat Lomuto's <= (M6). Insertion did *not* DNF at 10^5 (242 s): ties remove inversions (17.1M vs 25.2M comparisons at 10^4).
D Shift queue	Heap wins for $K \geq 50$; buffer wins only at $K = 10$ (my M5 crossover ≈ 16); full sort best only near $K = n/10$; C1 = C2; half the frauds by $K \approx 1{,}000$.	Heap won at *every* K, including 10 (**wrong**: per-record Python overhead, not comparisons, decides small K). **Wrong:** the full sort is still 3× slower than the heap at $n/10$; it wins only near $n/3$. C1 = C2 ✓. **Wrong:** half the frauds needs $K = 2{,}068$, and my $K = 100$ holds none (big Amounts first).
E Live feed	(b) heap fastest; (a) slowest and about 6× worse with 10-min windows; (c) in between; window size barely matters for (b).	(b) fastest ✓ (0.03 s / 0.06 s), (a) slowest ✓. **Wrong:** (a) grew 4.6×, not 6× (early re-sorts are small), and (c) got *faster* with smaller windows (1.09 to 0.56 s), which I had not expected; $n \lg(n/w)$ falls as w grows.
F M5/M6	M5 crossover $K \approx 16$; M6: comparisons $\approx \sum \binom{m_i}{2}$.	M5: tie at $K = 10$ by comparisons ✓, but time never favours the buffer. M6: within 0.012% at $n = 10^5$ ✓ (the remainder is $1.9n$ cross-band work).